Analysis Report: Blokus Occupied-Cell Cache PR Atomicity Experiment

Cross-judge synthesis of six LLM evaluations of PRs #52, #53, #55, #56, and #58

Prepared: 18 May 2026
Input files: GLM5.1.md, Opus46.md, Gemini3_Judge.md, KimiK25_Judge.md, GPT55_Judge.md, DeepSeekV4-Judge.md

Executive verdict. The experiment shows broad agreement that decomposing the occupied-cell cache feature into sequential PRs improved reviewability and isolated several meaningful risk classes. However, the judges diverged on whether every implementation agent cycle was justified. The strongest synthesis is: keep #53 and #58 as distinct risk units; move #58 before #55 or bundle it with #55; treat #56 as optional-separate depending on how much anchor-rule risk one wants to isolate; and consider bundling #52 with #53 when optimizing implementation-side cost.

1. Experiment Description

The experiment asked six judge models to evaluate whether five pull requests were appropriately atomic steps in one feature implementation: an occupied-cell cache for a Blokus game engine. Each judge evaluated PRs #52, #53, #55, #56, and #58 on three original criteria: coherence, rollback value, and distinct failure mode. A follow-up added a fourth criterion, cost proportionality, measuring whether a full separate implementation agent cycle was justified by the risk reduction, review clarity, and rollback value of that PR.

The report treats each uploaded Markdown transcript as one model judgment. It does not re-run repository analysis; it synthesizes the final scoring and reasoning contained in the six provided judge outputs. Scores are therefore interpreted as experimental observations, not as objective truth.

PR	Role in feature implementation
#52	Introduces the GameState.occupied_cells_by_player field, initialization, clone behavior, and serialization exclusion.
#53	Maintains the cache on the write path by updating it in apply_move after legal placements.
#55	Switches basic read helpers such as get_occupied_cells, is_first_move, and occupied_square_counts to cache-backed reads.
#56	Refactors anchor/move-generation helper logic to reuse cache-backed occupied sets.
#58	Rebuilds the derived cache on deserialization and adds round-trip/cache consistency safeguards.

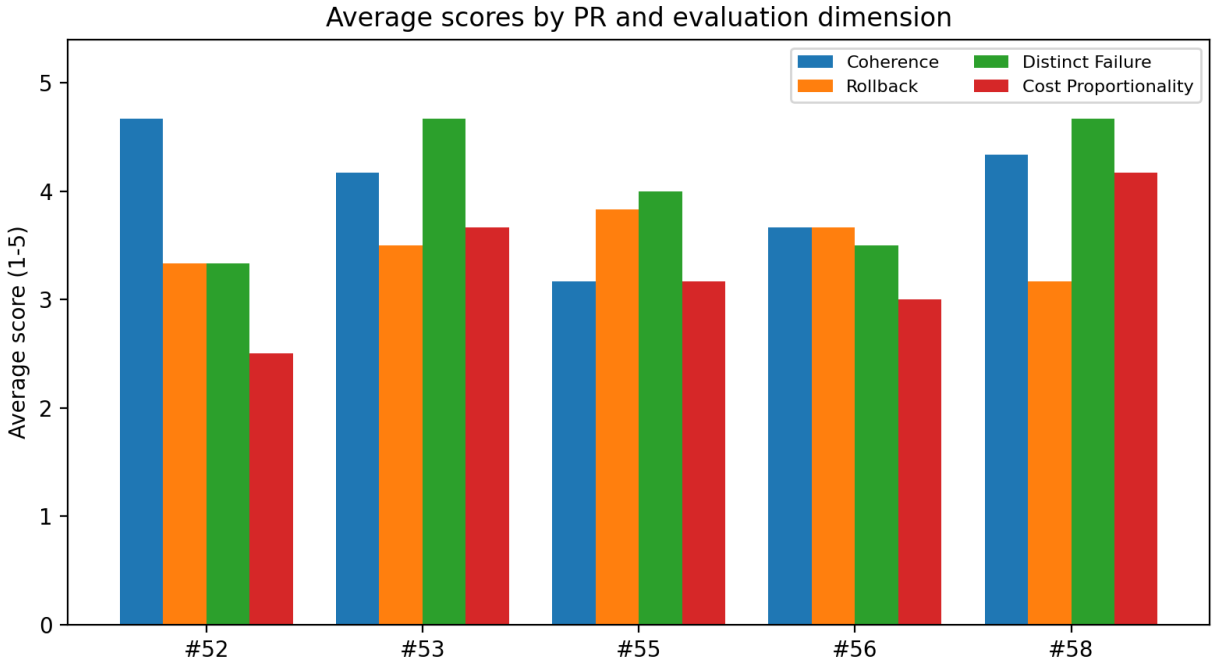
2. Methodology

The analysis used a comparative coding approach: final tables and rationales were extracted from each judge transcript, normalized into the same four scoring dimensions, and aggregated by mean, median, and standard deviation. Where a model gave a revised recommendation after cost-proportionality scoring, the revised recommendation was treated as the model's final process recommendation.

Interpretive caution: several judges used different assumptions about "merge safely alone." Some interpreted it as "after prior PRs but before later PRs," while others penalized PRs that exposed latent invariant gaps later repaired by #58. This explains most disagreement around #55 and #58.

3. Aggregated Results

PR	Title	Coherence μ	Rollback μ	Failure μ	Cost μ	Overall μ	Highest disagreement
#52	Initialize occupied-cells cache field	4.67	3.33	3.33	2.50	3.46	Cost (1.22)
#53	Update cache on apply_move	4.17	3.50	4.67	3.67	4.00	Rollback (1.22)
#55	Use cache for read helpers	3.17	3.83	4.00	3.17	3.54	Coherence (1.17)
#56	Refactor anchor helpers	3.67	3.67	3.50	3.00	3.46	Cost (1.26)
#58	Rebuild cache on deserialization	4.33	3.17	4.67	4.17	4.08	Rollback (1.72)


Average scores by PR and evaluation dimension
Coherence
Rollback
Distinct Failure
Cost Proportionality
Average score (1-5)
0
1
2
3
4
5
#52
#53
#55
#56
#58

4. Model-by-Model Score Matrix

Coherence

Coherence	#52	#53	#55	#56	#58
GLM-5.1	5	4	4	4	5
Opus 4.6	3	3	3	4	5
Gemini 3	5	4	3	4	1
Kimi K2.5	5	5	5	4	5
GPT-5.5	5	5	2	4	5
DeepSeek V4	5	4	2	2	5

Rollback

Rollback	#52	#53	#55	#56	#58
GLM-5.1	3	4	4	3	5
Opus 4.6	2	2	5	4	2
Gemini 3	5	4	4	4	5
Kimi K2.5	4	5	4	3	4
GPT-5.5	3	4	4	5	2
DeepSeek V4	3	2	2	3	1

Distinct Failure

Distinct Failure	#52	#53	#55	#56	#58
GLM-5.1	4	5	4	3	5
Opus 4.6	4	5	4	3	5
Gemini 3	2	5	4	4	5
Kimi K2.5	3	4	4	3	5
GPT-5.5	4	5	4	4	5
DeepSeek V4	3	4	4	4	3

Cost Proportionality

Cost Proportionality	#52	#53	#55	#56	#58
GLM-5.1	2	3	3	2	5
Opus 4.6	2	2	3	2	3
Gemini 3	1	4	4	5	4
Kimi K2.5	4	5	4	2	5
GPT-5.5	4	5	3	3	4
DeepSeek V4	2	3	2	4	4

5. Findings

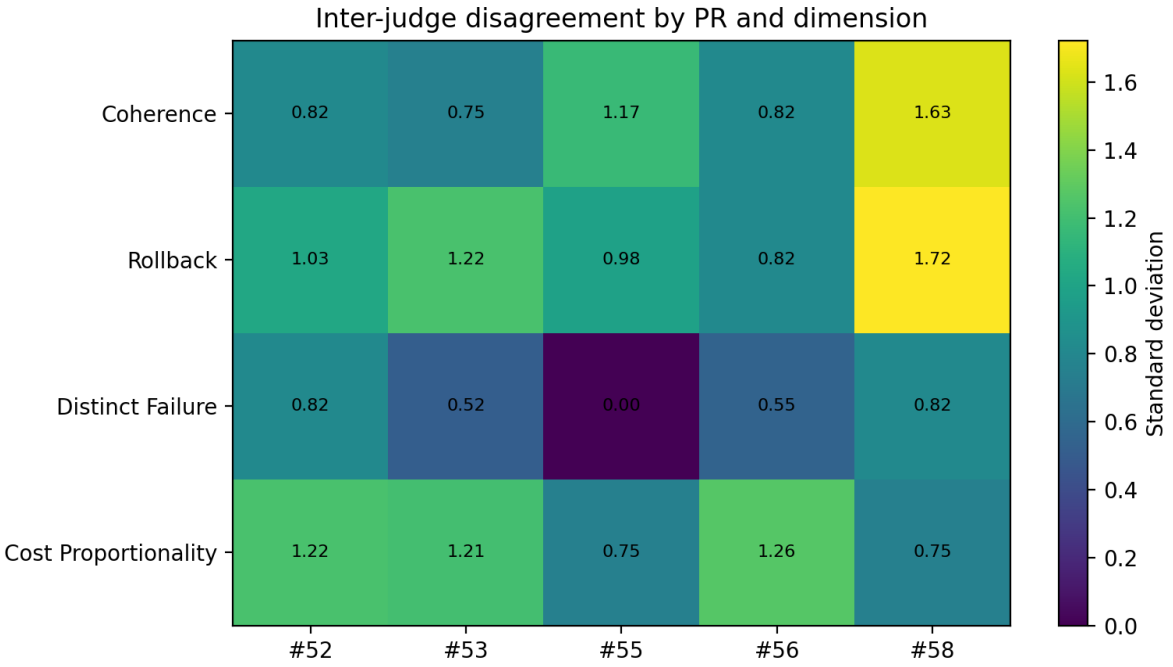
Finding 1: PR #53 is the clearest atomic implementation step. Across judges, #53 received high distinct-failure scores because write-path cache maintenance is the central cache-consistency risk. It cleanly isolates whether legal moves update the derived cache and whether illegal moves avoid mutation.

Finding 2: PR #58 is technically distinct but was often judged as late. Most judges recognized serialization/deserialization as a unique persistence-boundary risk. The key criticism was ordering: read helpers started trusting the cache in #55 before deserialized states reliably rebuilt it.

Finding 3: PR #55 is the main sequencing dispute. Some judges scored #55 highly because it is a coherent read-path refactor after #52 and #53. Others penalized it because cache-backed reads could be wrong for serialized round-trips until #58 landed.

Finding 4: PR #56 is the cost-proportionality controversy. Judges agreed that anchor logic has real behavioral risk, but disagreed on whether the actual implementation diff justified a full separate implementation cycle. This makes #56 a process-policy decision rather than a purely technical one.

Finding 5: PR #52 is coherent but not always worth a standalone agent run. The setup PR is safe and easy to reason about, but several judges viewed it as a checkpoint-sized scaffold that should be bundled with #53 when implementation-side overhead matters.



6. Per-PR Interpretation

#52 — Initialize occupied-cells cache field. Strong standalone coherence; weaker cost proportionality. It establishes the derived-state contract, but by itself has limited functional value. Best bundled with #53 if reducing implementation-cycle overhead is important.

#53 — Update cache on apply_move. Best-supported separate PR. It isolates write-path correctness and has a clear failure mode: cache-board drift during move application. Keep separate.

#55 — Use cache for read helpers. Substantive read-path refactor, but should not precede complete cache-invariant coverage. Best either after #58 or bundled with #58.

#56 — Refactor anchor helpers. Acceptable separate PR only if the team explicitly wants isolated validation of anchor/move-generation behavior. Otherwise bundle with #55 as part of cache-backed read optimization.

#58 — Rebuild cache on deserialization. Strongly distinct serialization-boundary PR. Keep separate in principle, but sequence it before #55 or merge it into #55 if maintaining an always-valid intermediate state is mandatory.

7. Final Recommendation

PR	Synthesis recommendation	Reason
#52	Bundle with #53 unless educational/review traceability is prioritized.	It is safe, but the implementation-side cost is weak relative to its standalone functional value.
#53	Keep separate.	Write-path cache maintenance is the most central and cleanly isolated risk unit.
#55	Move after #58 or bundle with #58.	Read paths should not trust a derived cache until all construction paths maintain it.
#56	Conditional: keep separate only for anchor-rule risk isolation; otherwise bundle with #55.	Anchor generation is domain-sensitive, but the implementation is narrow.
#58	Keep separate, but reorder before #55.	Serialization is a distinct boundary risk and should close the invariant before cache-backed reads expand.

Recommended implementation sequence: #52 + #53 as one passive-cache checkpoint; #58 to complete state construction invariants; #55 to switch basic reads; #56 as either a bundled optimization within #55 or a separate anchor-risk PR.

8. Confidence and Limitations

Confidence level: 0.87. The main synthesis is robust because the uploaded judge outputs repeatedly identify the same technical risk categories: data-model initialization, write-path cache maintenance, read-path substitution, anchor generation, and deserialization. Confidence is below 0.95 because the transcripts do not all apply identical assumptions for standalone mergeability, and some score tables changed during follow-up cost-proportionality evaluation.

Limitations: this report synthesizes judge outputs rather than independently verifying every repository diff. It should be used as an experiment-analysis artifact, not as a substitute for direct code review of the current repository state.

Appendix A: Normalized Score Dataset

Model	PR	Coherence	Rollback	Failure	Cost
GLM-5.1	#52	5	3	4	2
GLM-5.1	#53	4	4	5	3
GLM-5.1	#55	4	4	4	3
GLM-5.1	#56	4	3	3	2
GLM-5.1	#58	5	5	5	5
Opus 4.6	#52	3	2	4	2
Opus 4.6	#53	3	2	5	2
Opus 4.6	#55	3	5	4	3
Opus 4.6	#56	4	4	3	2
Opus 4.6	#58	5	2	5	3
Gemini 3	#52	5	5	2	1
Gemini 3	#53	4	4	5	4
Gemini 3	#55	3	4	4	4
Gemini 3	#56	4	4	4	5
Gemini 3	#58	1	5	5	4
Kimi K2.5	#52	5	4	3	4
Kimi K2.5	#53	5	5	4	5
Kimi K2.5	#55	5	4	4	4
Kimi K2.5	#56	4	3	3	2
Kimi K2.5	#58	5	4	5	5
GPT-5.5	#52	5	3	4	4
GPT-5.5	#53	5	4	5	5
GPT-5.5	#55	2	4	4	3
GPT-5.5	#56	4	5	4	3
GPT-5.5	#58	5	2	5	4
DeepSeek V4	#52	5	3	3	2
DeepSeek V4	#53	4	2	4	3
DeepSeek V4	#55	2	2	4	2
DeepSeek V4	#56	2	3	4	4
DeepSeek V4	#58	5	1	3	4